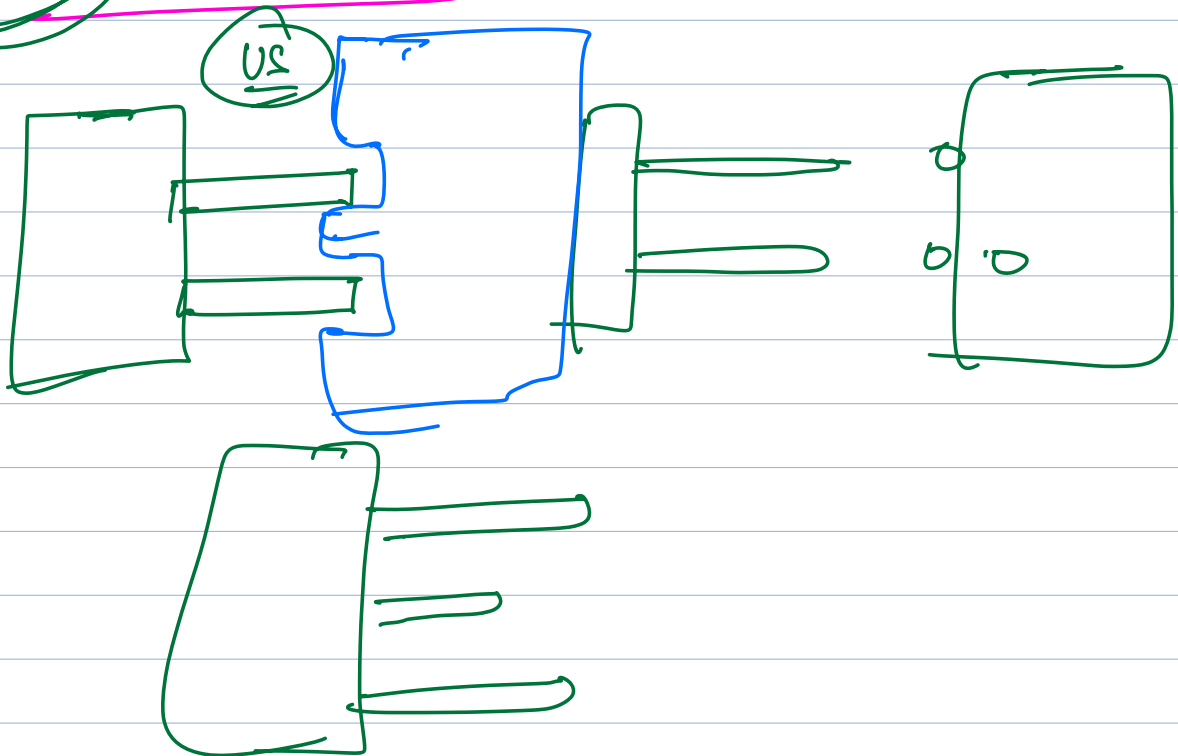


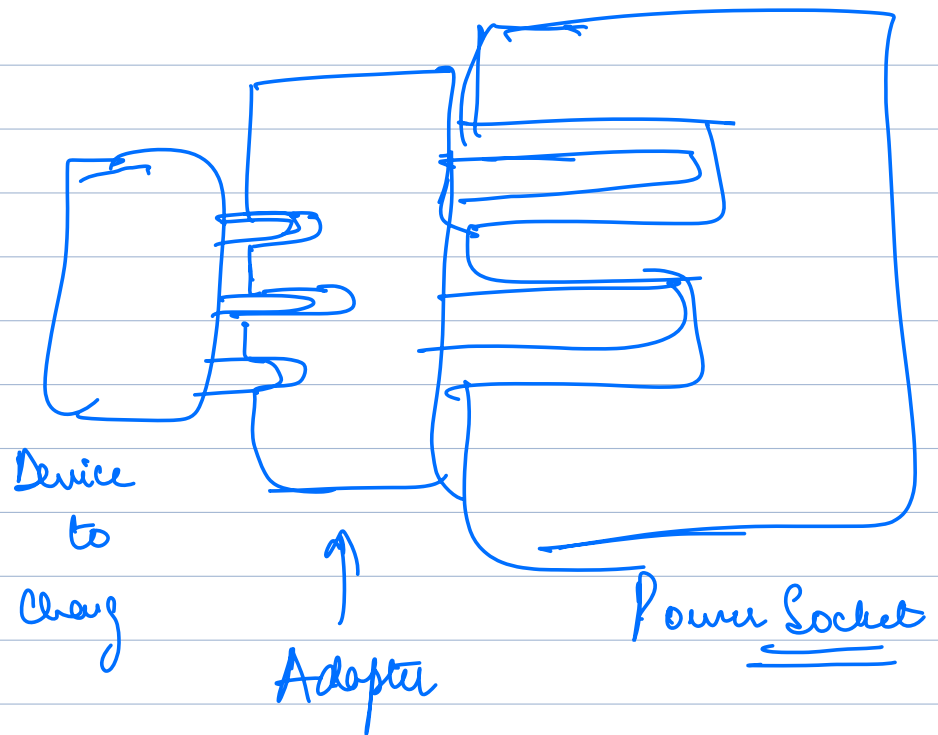
Structural Design Patterns

- ① Adapter
- ② Facade

Will start
at 9:10PM

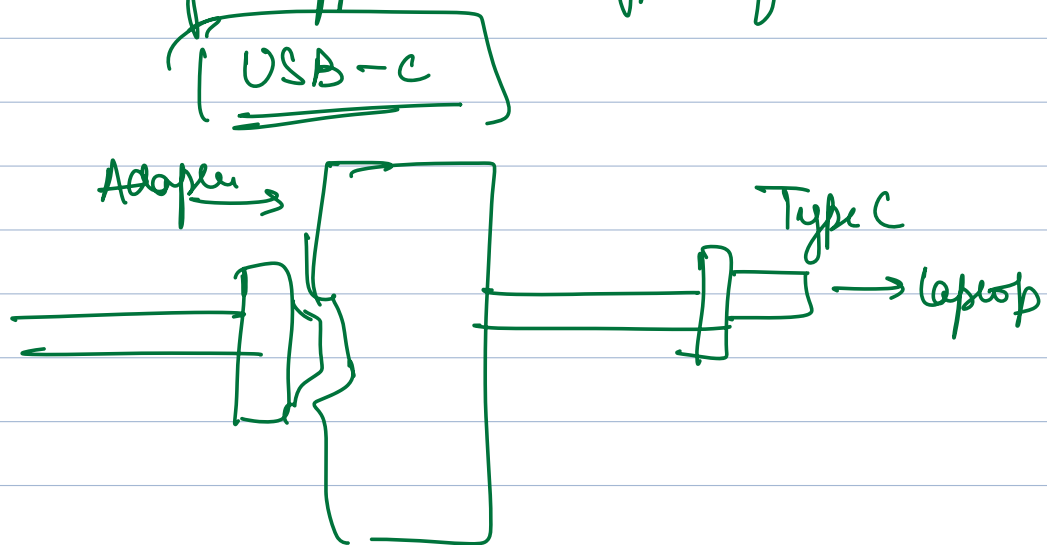
⇒ Converter
Adapter Design Pattern





(allows to convert
1 type of thing to
another type of thing)

Macbook only supports one type of port



Why Adapter

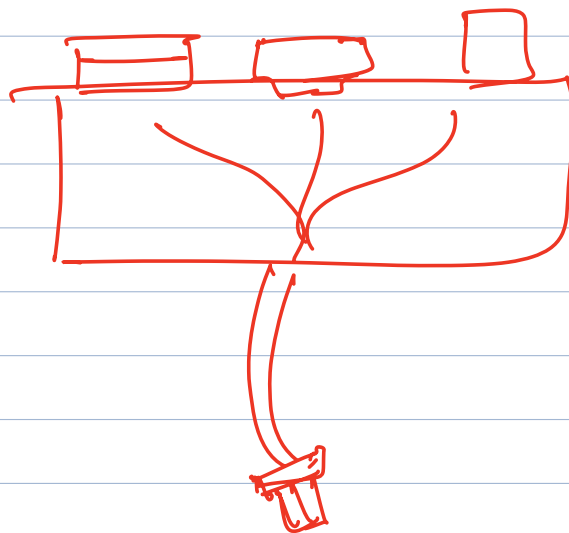
If adapters weren't there →

- ① something universal
- ② multiple types of sockets.

⇒ It's not practical to have multiple types of sockets

If apple wanted to support all types of ports, its SWE will have to write code to read from diff types of devices

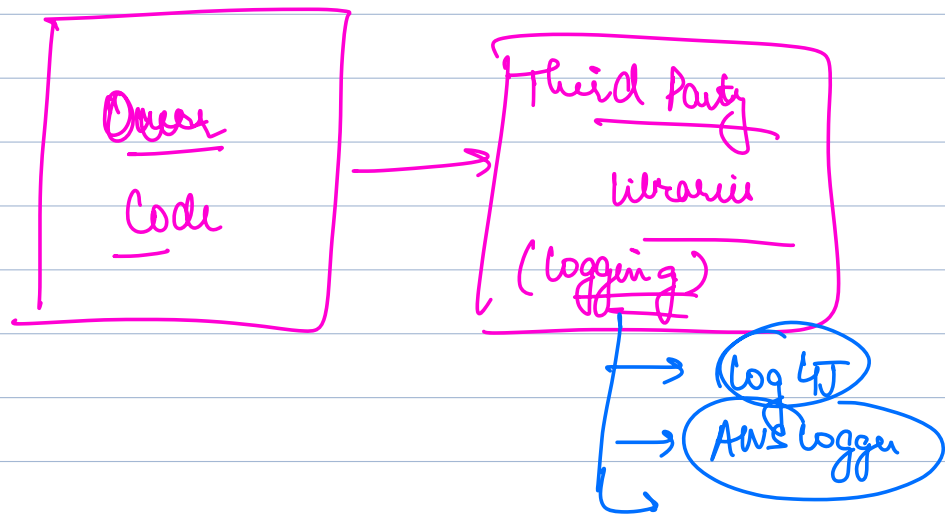
Y/S if I had an adapter → write code to read from only one type of device
⇒ USB-C



⇒ adapter also converts

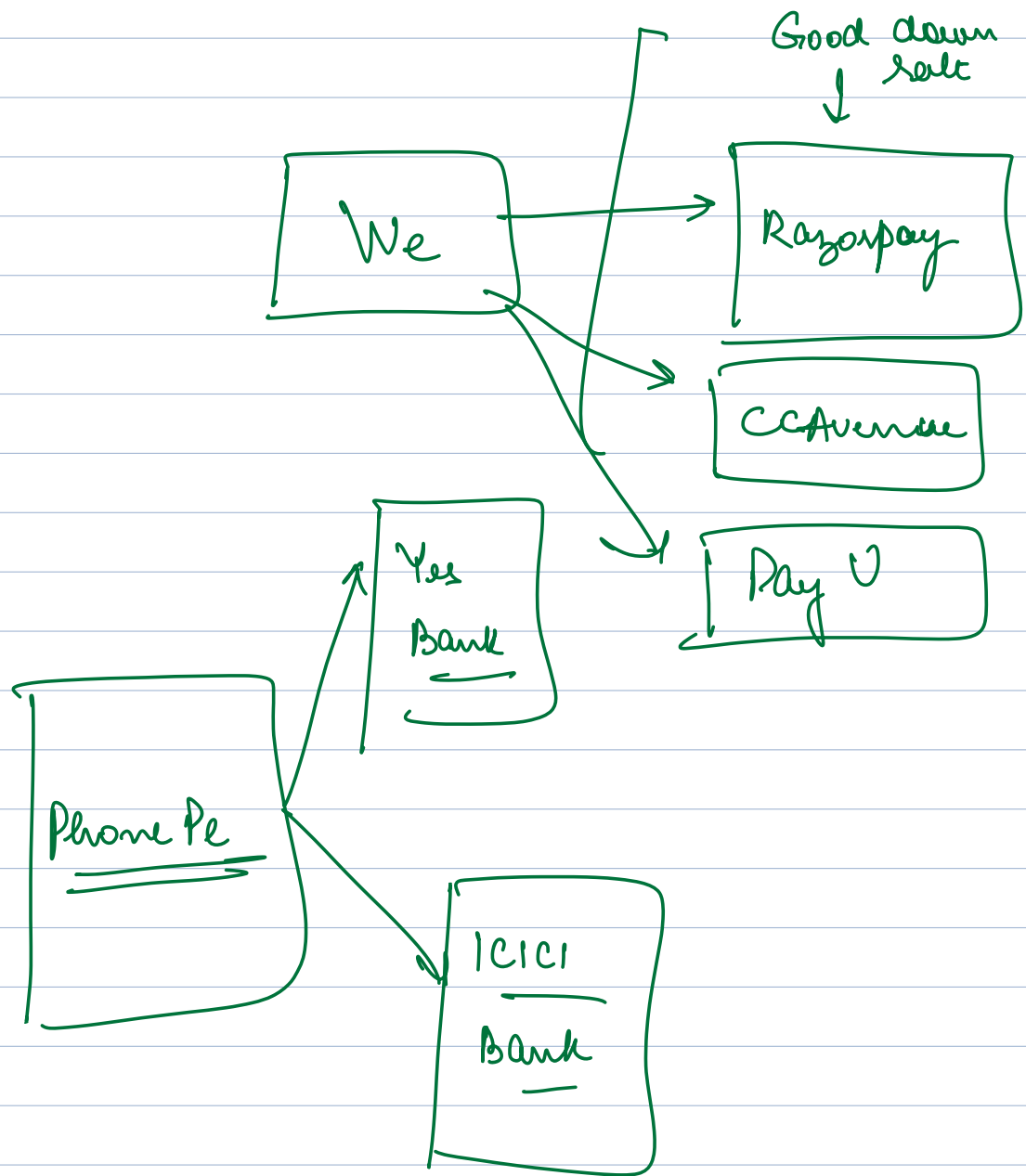
Adapter Benefits

- ① You only have to support one type of thing
- ② Handles conversion from other type of things to things you need.



Problem Statement

- ① I am connecting to 3rd party codebase
- ② I might want to connect to another type of that
- ③ library might be deprecated (it may no longer be supported)
- ④ We might want to allow plug and play

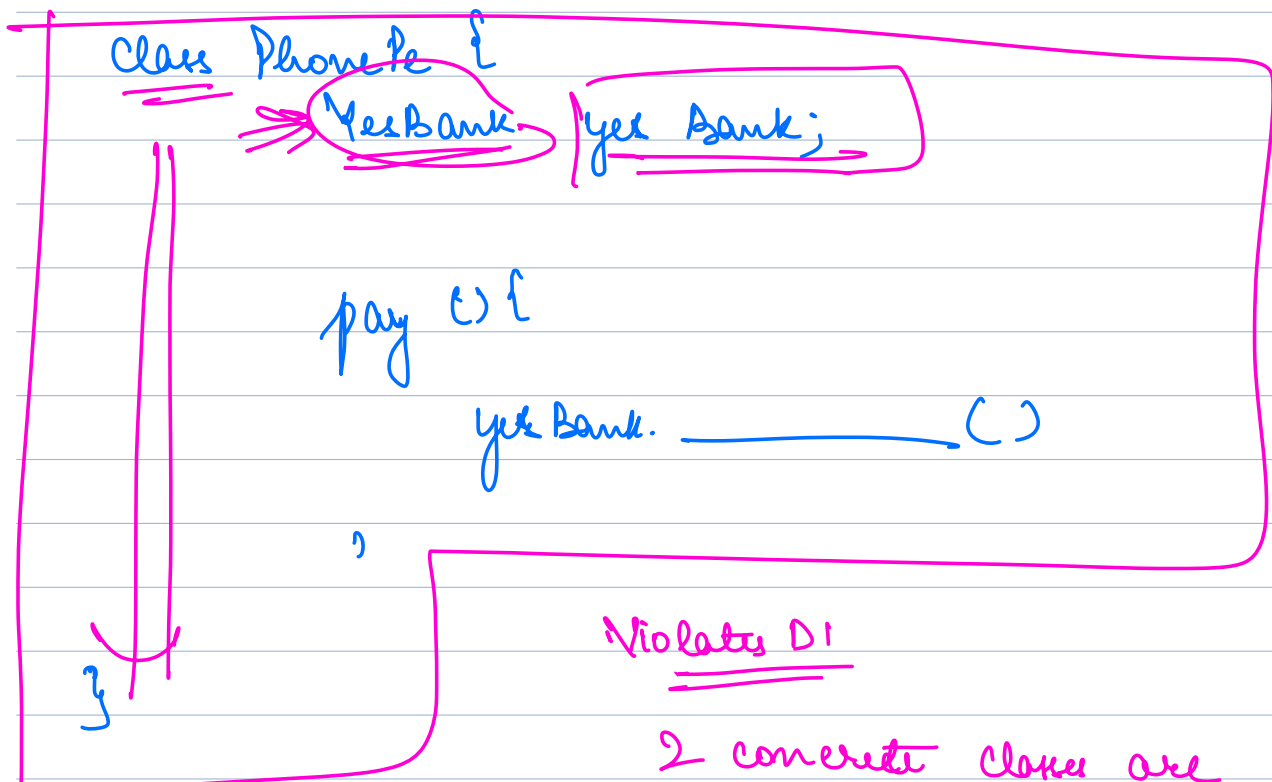
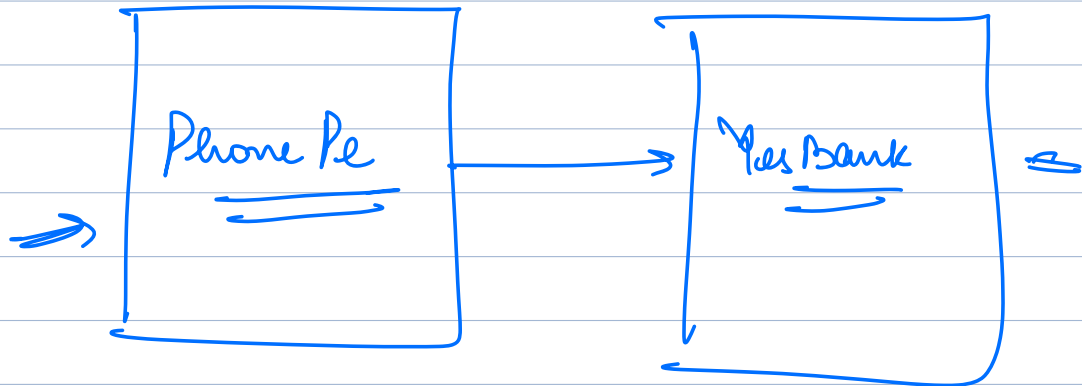


Whenever you have to connect to a 3rd party library, how will you structure your codebase?

- Maintainable
- Extensible

How?

①



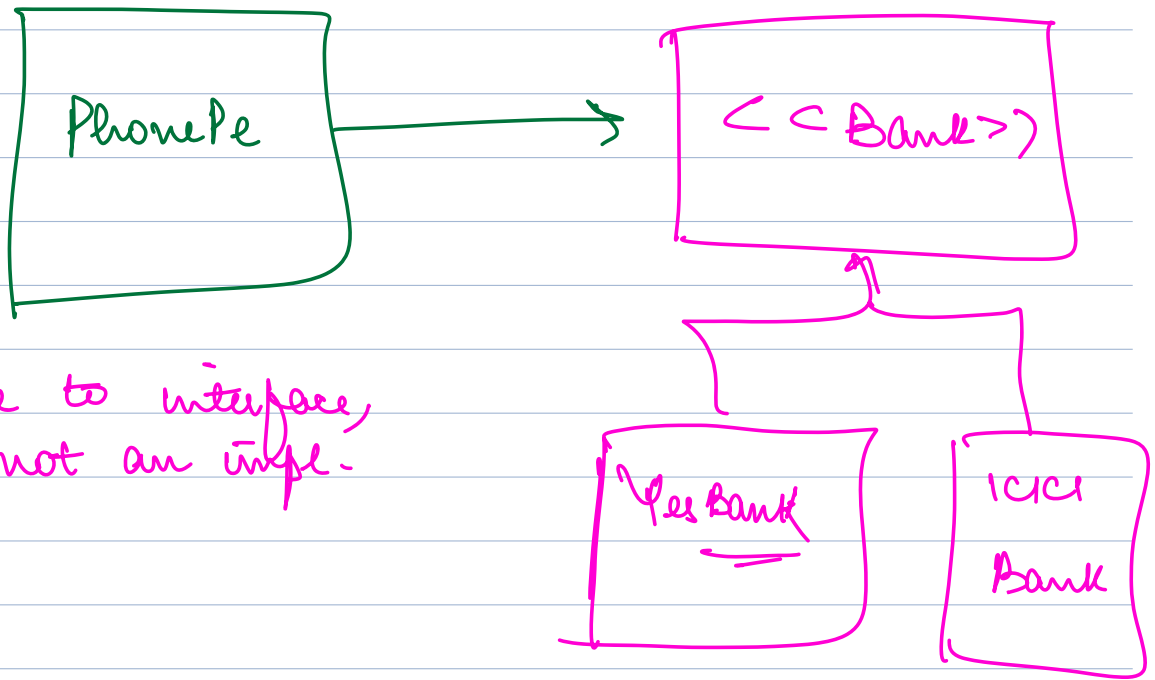
Violates DI

2 concrete classes are directly talking to each other.

Why Bad

Higher tightly coupled, not easy to change.

⇒ Instead of connecting 2 classes directly, we should connect via an interface.



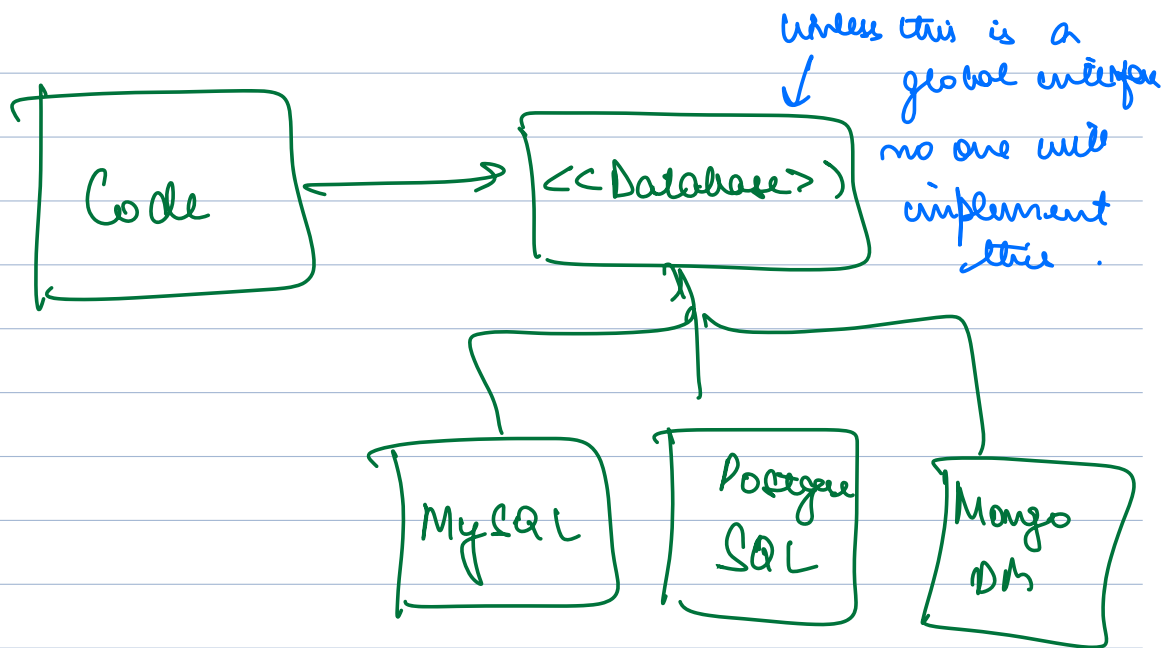
⇒ Code to interface, not an impl.

```
PhonePe {  
    Bank b = new ICICIBank();  
}
```

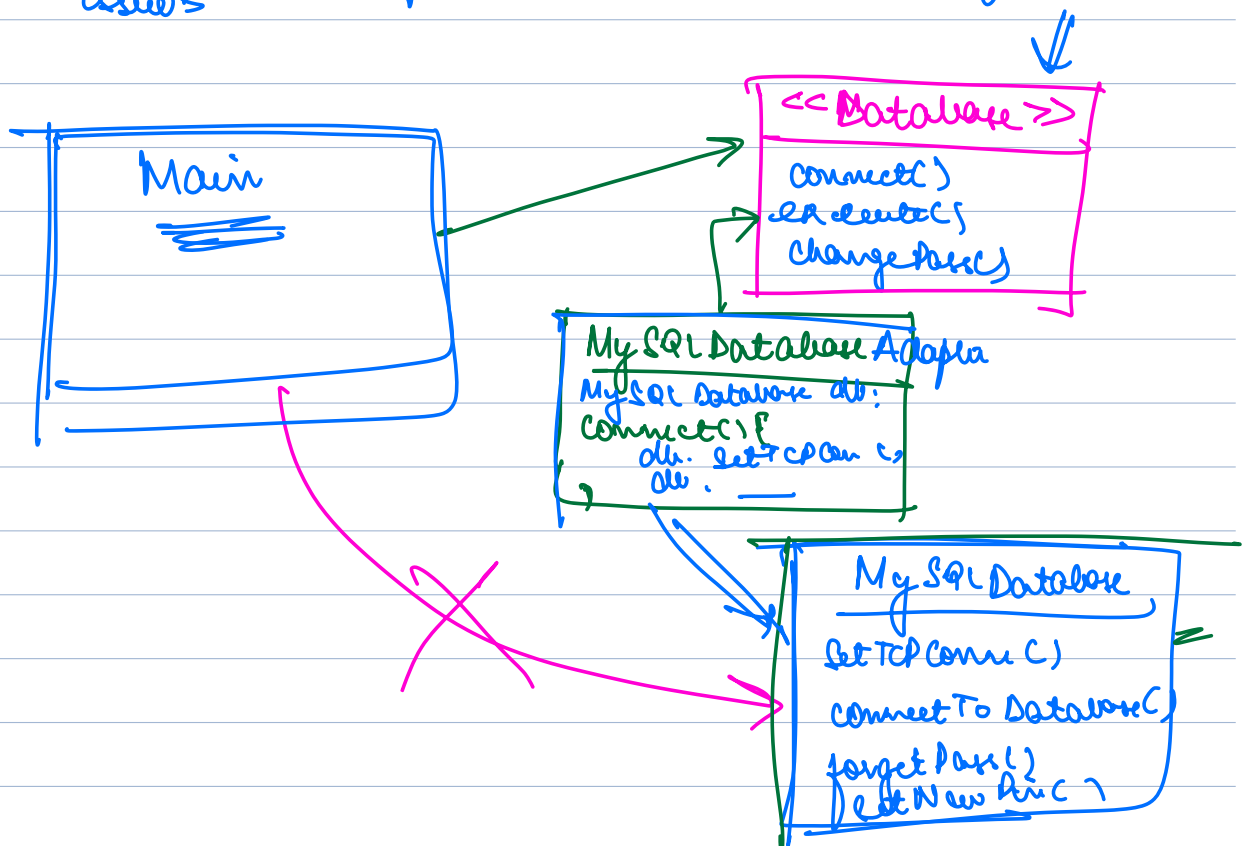
pay() {

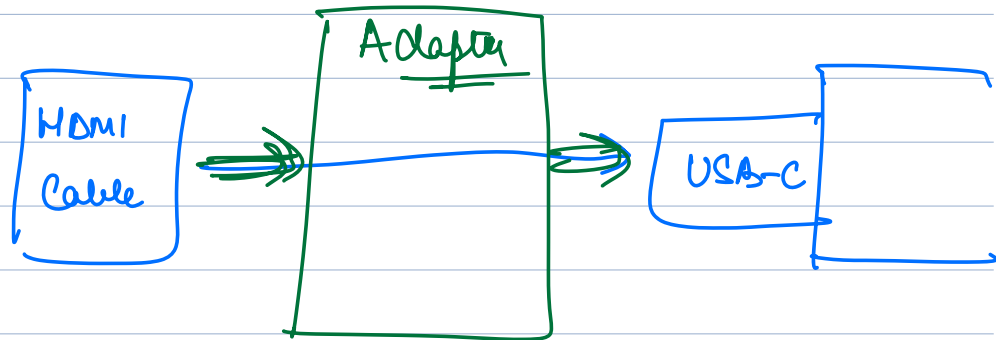
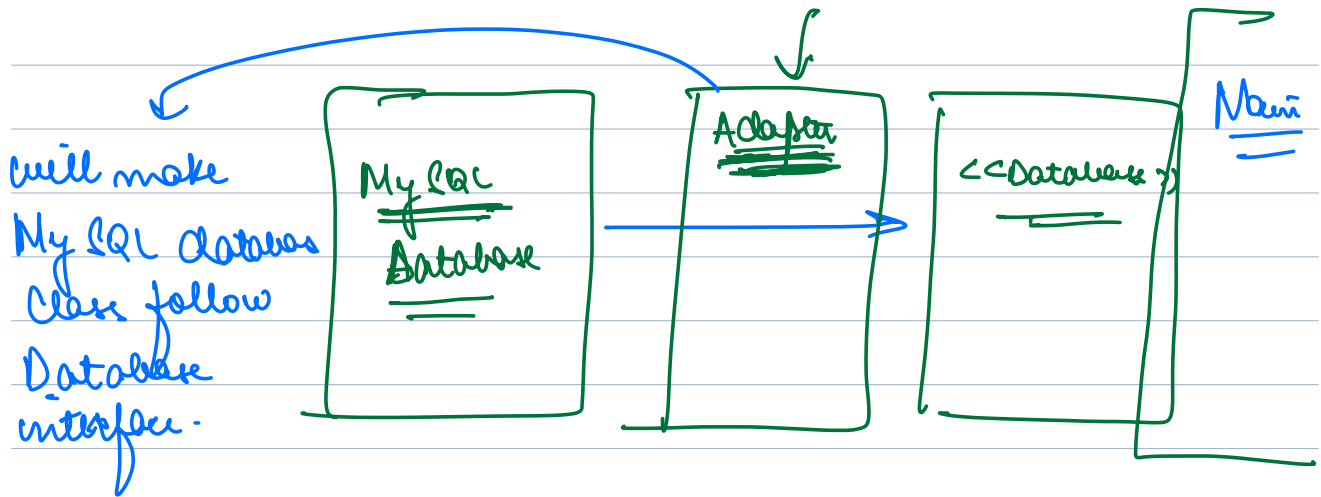
p. do something()

}



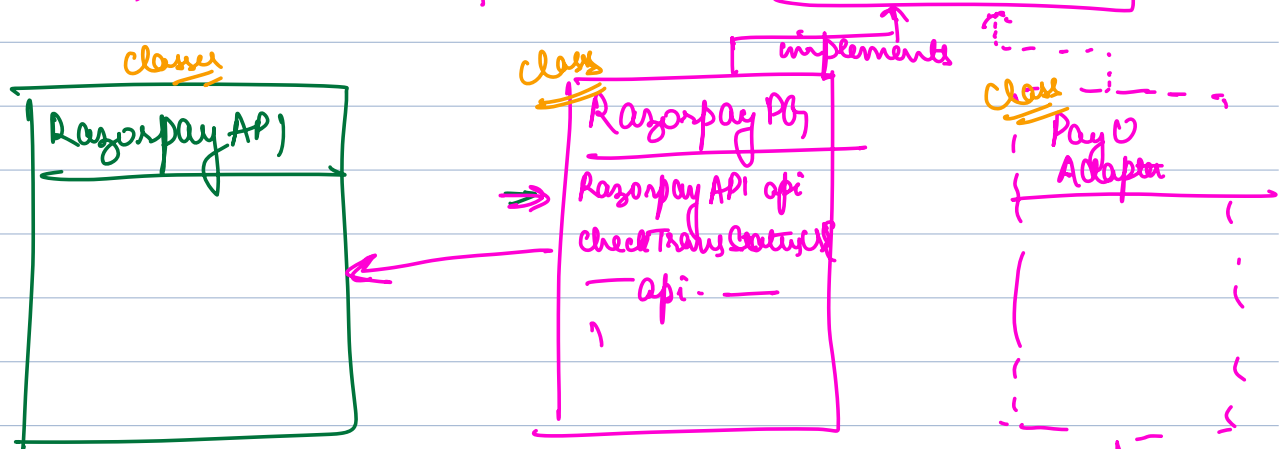
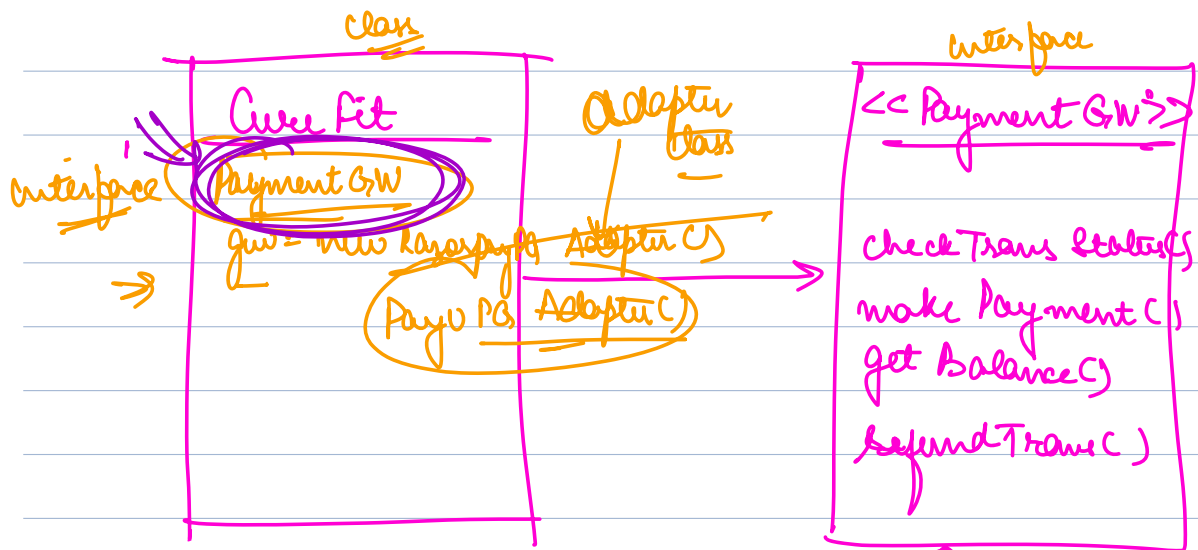
⇒ if no standard interface, I should still not code directly to classes because of extensibility issues.





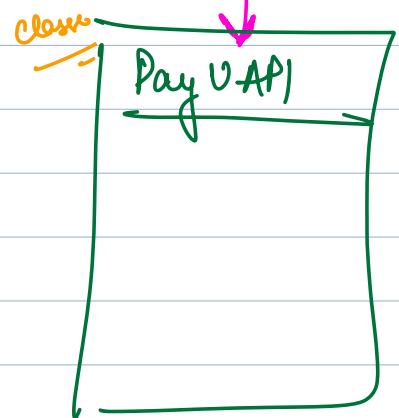
Whenever you are connecting to a 3rd party library, don't connect to it directly as it violates DI principle. Instead connect to it via an interface in slw.

- ① Create an interface with methods that you need from the third party.



→ 3rd Party

→ U.S.



→ follows DI

→ follows OCP

```
class Payment GW factory {
```

```
    Payment GW    getPayment GW from Name  
                  (String name) {
```

```
        if (name == ____ )
```

```
            return new Razorpay PG  
                  Adapter();
```

```
        else
```

```
            return new PayU PG Adapter();
```

```
}
```

Contexts



① ⇒ OOP, Java, Conc

② ⇒ Design Principles, Patterns

class A {

→ b(c) {
 A.lock
 A.unlock()

lock() ≠ new
 Reentrant
 lock()

→ c(c) {
 } T1

→ d(c) {
 A.lock() ✓

 A.unlock()

→ T2

}